

第5章 散列

建议学时:4-5 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言,DS&A 零基础(已读完第 1-4 章)

🎯 本章学习目标

- 理解散列为什么能把查找从 $O(\log n)$ 压到平均 $O(1)$:散列函数把键映射为数组下标,理想情况下一步直达。
- 掌握整数取模与字符串 37 倍多项式两种散列函数,能说出"为什么表大小必须是素数、为什么要最后才取模"。
- 掌握分离链接法与开放定址法(线性探测、平方探测、双散列)的实现,理解装填因子 λ 与惰性删除。
- 掌握再散列的动机与实现,能说出摊还代价 $O(1)$;了解可扩散列的基本思想。
- 深入理解 dict:内建散列表,可哈希要求、插入顺序、推导式与 Counter 等常用写法。
- 能独立实现词频统计程序(手写散列表 + dict 两条路线),用实验数据理解装填因子的影响。

💡 从 C 到 Python:本章主题如何迁移

数组按下标访问是 $O(1)$ 的,前提是下标是 0、1、2、... 这样的连续整数。可现实里的键往往不连续:学号是 2023001、2023050,订单号是一串字母数字。散列的想法很简单:写一个函数把"任意的键"翻译成"一个合理的下标",然后直接进数组。这个函数就叫 **散列函数**。

C 语言里你要自己写这个函数,并处理两个麻烦:负数取模会得到负下标(必须 `if (h < 0) h += M`),字符串散列时 int 容易溢出。Python 里这两个麻烦都不存在:取模向下取整、余数恒非负,整数任意精度不会溢出——但"散列思想"本身一字不少,本章的实验仍然要手写一张表。

更重要的差异是:Python 的 dict 就是一张工程级的散列表,你每天都在用而浑然不觉。本章的主线是"先手写,再用库":\$5.2 用 list of list 拼出分离链接散列表(桶就是链表,是 C 语言"结构体 + 指针"的 Python 化);\$5.6 对照着讲 dict 的底层机制——可哈希、扩容、插入序;\$5.7 的单词统计证明手写版与 dict 输出完全一致。

📦 前置知识自查

数学前置:模运算(取余)。重点:Python 的 `%` 是向下取整除法,结果与 C 不同(见 \$5.1.2 误区)。另一条是素数:只能被 1 和自身整除的正整数。

C 语言前置:指针遍历字符串、struct 与链表节点、malloc/free。本章的 C 对照表会反复出现这三样,帮你看清 Python 省掉了什么。

依赖的前面章节:第 3 章链表与 vector——分离链接法的桶就是链表;再散列"翻倍扩容再重插"与 Python list 扩容、dict 扩容同构。

🚀 本章学习路线

1. **第一阶段:**读 \$5.1-\$5.2,吃透散列函数与分离链接法;手算教材 5.2 节小例子,对照示例 5-3 抄一遍代码。
2. **第二阶段:**读 \$5.3-\$5.5,理解三个探测方案的"聚集"问题与平方探测定理;读 \$5.6 时在解释器里逐个敲 dict 示例。
3. **第三阶段:**完成章末实验,手写表与 dict 双路线统计词频,用计时数据看 λ 对性能的影响。

本章知识导图

- §5.1 基本想法与散列函数
 - 5.1.1 基本想法:查找的第三件武器
 - 5.1.2 整数键:取模散列与素数表大小
 - 5.1.3 字符串键:37 倍多项式散列
 - 5.1.4 装填因子 λ
- §5.2 分离链接法
 - 5.2.1 桶就是一条链表
 - 5.2.2 完整实现:查找、插入与删除
 - 5.2.3 复杂度:平均 $O(1)$ 从哪来
- §5.3 开放定址法
 - 5.3.1 线性探测与一次聚集
 - 5.3.2 平方探测
 - 5.3.3 双散列
 - 5.3.4 三种方案对比与惰性删除
- §5.4 再散列
 - 5.4.1 什么时候需要再散列
 - 5.4.2 实现:翻倍、取素数、重插
 - 5.4.3 摊还代价
- §5.5 可扩散列(选读)
- §5.6 标准库中的散列结构
 - 5.6.1 dict:Python 内建的散列表
 - 5.6.2 可哈希:什么对象能当键
 - 5.6.3 常用写法:推导、defaultdict 与 Counter
- §5.7 手写与标准库并排:单词统计

§5.1 基本想法与散列函数

5.1.1 基本想法:查找的第三件武器

你学过两种查找:顺序扫描(链表, $O(n)$)和二叉查找树(平均 $O(\log n)$)。散列是第三条路:把每个键换算成数组下标,查找就是一次数组访问,平均 $O(1)$ 。

表大小为 M ,散列函数 `hash(x)` 把键 x 映射到 0 到 $M-1$ 。插入: `table[hash(x)] = x`;查找:查 `table[hash(x)]` 里有没有 x 。难点在于散列函数"多对一":不同键可能映射到同一下标,这叫 **冲突**。散列的学问 = 好的散列函数(冲突少)+ 好的冲突解决策略。

记住 **散列三要素**:散列函数(键 $\rightarrow$ 下标)+ 冲突解决策略(§5.2 分离链接 / §5.3 开放定址)+ 表大小 M (尽量取素数)。任一环做差,性能都会从"平均 $O(1)$ "掉回"最坏 $O(n)$ "。

5.1.2 整数键:取模散列与素数表大小

键是整数时最常用的散列函数是取模: $\text{hash}(x) = x \% M$, 开销只有一次除法。关键是表大小 M 的选择: 若 M 是合数而键恰好都带某个公因子, 就会全挤进少数几个桶。比如 $M = 10$, 键 42、32、52、72 的个位全是 2, $x \% 10$ 全部等于 2——四键同桶, 九桶全空; 取素数 $M = 11$ 则余数 9、10、8、6, 四散而开。键的分布未知时, 素数表是唯一稳妥的选择。

 **误区: Python 的 % 与 C 不同——向下取整**

C 的 `-5 % 7` 等于 -5, Python 的 `-5 % 7` 等于 2(向下取整, 余数恒非负)。对散列来说这是好事: 下标天然合法, C 里那行 `if (h < 0) h += M` 在 Python 里永远用不上。但移植 C 代码时务必警惕——同样的表达式, 两种语言给出不同结果:

```
# Python 的 % 与 C 不同: 向下取整, 余数恒为非负
x = -5
M = 7
h = x % M                # 结果是 2, 不是 -5!
print(h)                 # 输出 2, 天然适合当数组下标
# 因此 C 里那行 if (h < 0) h += M 在 Python 里永远用不上
# 但移植 C 代码时要警惕: 两种语言对负数取模语义不同
# 若 C 代码里散列值出现负数, 先怀疑取模语义再查逻辑
```

另一个顺带的好处: Python 整数任意精度, 散列累加永远不会溢出——C 里 `int` 溢出导致的“散列值乱跳”问题, 在 Python 里不存在。

C 的写法	Python 的写法
<code>int h = x % M;</code> 负数得到负下标, 要手动 <code>if (h < 0) h += M;</code>	<code>h = x % M</code> 向下取整, 结果恒非负, 坑直接消失
<code>char*</code> 循环累加 <code>h = h*37 + *s</code> , <code>int</code> 可能溢出	<code>h * 37 + ord(c)</code> , 整数任意精度, 无溢出

5.1.3 字符串键:37 倍多项式散列

字符串是最常见的键(单词、文件名、URL)。教材采用 37 倍多项式法, 把字符串按霍纳法则逐位乘加, 每步 `h = h * 37 + c`。先给 C 回顾版, 再给 Python 写法:

示例 5-1 C 回顾: 字符串散列(37 倍多项式, 霍纳法则)

```
/* C 回顾: 字符串散列 */
int hashStrC(char* s, int M) {
    unsigned int h = 0;                /* unsigned 累加, 防止溢出成负数 */
    while (*s != '\0') {
        h = h * 37 + (unsigned char)*s; /* 每步乘 37 再加当前字符 */
        ++s;
    }
    return (int)(h % M);                /* 最后才取模, 结果落在 [0, M-1] */
}
```

示例 5-2 Python 版: 同样的逻辑, 更短的代码

```
# Python 版:整数任意精度,不会溢出;逻辑与 C 完全对应
def hash_str(s: str, M: int) -> int:
    h = 0
    for c in s:
        # 逐个字符取码,累乘累加
        h = h * 37 + ord(c) # 每一步都乘 37 再加当前字符
    return h % M # 最后取一次模,结果在 [0, M-1]

# 调用示例:h = hash_str("hello", 17) -> 0 到 16 之间的整数
```

两个版本除类型与遍历方式外一字不差;"最后取一次模"几个字不是风格,是正确性——见下面的误区。

⚠ 误区:逐字符取模会悄悄毁掉散列

有人图省事,把取模写进循环:

```
# 错误示范:把取模写进循环,乘法被取模"吃掉"
for c in s:
    h = (h * 37 + ord(c)) % M # 错误!
# 当 M = 37 时,(h*37+c)%37 = c%37,
# 散列退化成只看最后一个字符;
# 当 M = 36 时,37 ≡ 1 (mod 36),
# "abc" 与 "cba" 算出同一个散列值,
# 字符串变成了"字符和"。
```

两种退化同源:取模发生在乘法之前,丢掉了进位。正确写法永远是先累加、最后取一次模(示例 5-2)。

5.1.4 装填因子 λ

装填因子 λ = 元素个数 N / 表大小 M ,衡量表"满"的程度: λ 越大冲突越多,查找越慢; λ 越小空位越多,浪费越大。分离链接的经验阈值 $\lambda \approx 0.75$ (再大就该扩容,见 §5.4);开放定址更苛刻,平方探测要求 $\lambda < 1/2$ 。

🚀 记号说明: λ 的两种记法

正文用希腊字母 λ ;代码里命名为 `load_factor` (或 `loadFactor()`),实现为 N / M 的浮点除法,两者是同一个量。

§5.2 分离链接法

5.2.1 桶就是一条链表

分离链接法(separate chaining)的思路最直观:每个下标放一条链表(教材图 5-1)。插入:算桶号挂到链表;查找:算桶号,链内线性扫描;删除:算桶号,链中摘除。 M 决定链表条数, $\lambda = N/M$ 就是平均链长。Python 里"链表"用 `list` 代替,桶数组就是 `list of list`。

C 的写法	Python 的写法
<pre>struct Node { char* key; Node* next; }; 手写链表头插法,自己 malloc/free 每个节点</pre>	桶 = list,桶数组 = list of list,一行 <code>[[] for _ in range(M)]</code>
散列后用 <code>strcmp</code> 逐个比较	桶内元素直接 <code>==</code> 比较
手动 free 所有节点	交给垃圾回收,无释放概念

5.2.2 完整实现:查找、插入与删除

下面是分离链接散列表的完整 Python 实现。 `next_prime(M)` 返回不小于 M 的最小素数(实现见章末实验),构造函数用它定素数表大小。

示例 5-3 分离链接散列表:find / insert / remove

```
# 分离链接散列表:桶数组是 list of list,一个桶就是一条链表
class HashTable:
    def __init__(self, M):
        self.buckets = [[] for _ in range(next_prime(M))]
        self.size = 0

    def _hash(self, s):
        h = 0
        for c in s:
            h = h * 37 + ord(c)
        return h % len(self.buckets)

    def find(self, s):
        for x in self.buckets[self._hash(s)]:
            if x == s:
                return True
        return False

    def insert(self, s):
        if self.find(s):
            return
        self.buckets[self._hash(s)].append(s)
        self.size += 1

    def remove(self, s):
        b = self.buckets[self._hash(s)]
        for i, x in enumerate(b):
            if x == s:
                b.pop(i)
                self.size -= 1
        return

    @property
    def load_factor(self):
        return self.size / len(self.buckets)
```

三个操作同一套路:先 `_hash(s)` 定桶,再只动那一条链——散列把"全表扫描"降为"单桶扫描",单桶平均只有 λ 个元素。insert 先查重再 append;remove 定位后 `pop(i)` 摘除。与 C 的差异在桶上:Python 的 list 自带增删,免去全部指针操作。

5.2.3 复杂度:平均 $O(1)$ 从哪来

一次 find 的代价 = 散列 $O(\text{键长 } L)$ + 单桶扫描 $O(\text{平均链长}) = O(1 + \lambda)$ 。 λ 控制在 0.75 左右时,常数 $1 + \lambda$ 很小,于是散列表查找 **平均 $O(1)$** ;插入、删除同理。

牢记散列表的复杂度画像:查找/插入/删除平均 $O(1)$ 、最坏 $O(n)$ 。最坏情况是所有键散列到同一桶,链表退化成一条长链——散列函数均匀且表大小是素数时几乎不会发生,但你必须能说出它为什么是 $O(n)$ 。代价是牺牲顺序性:无法按键

有序遍历、没有前驱后继;要顺序时回到第 4 章的二叉查找树。

§5.3 开放定址法

分离链接需要额外指针,浪费空间。开放定址法(open addressing)让所有元素都住进表本身:冲突时按某种规则在表中找下一个空位。规则不同,衍生出三种方案。

5.3.1 线性探测与一次聚集

线性探测:冲突时逐个向后试 $pos = (pos + 1) \% M$,直到空位或绕回。它有个著名毛病——一次聚集:连续被占的单元越滚越长,新键插进去又让它更长,形成"雪球"。

示例 5-4 线性探测插入

```
# 开放定址:线性探测插入(空单元用 None 标记)
pos = hash_str(s, M)
while table[pos] is not None and table[pos] != s:
    pos = (pos + 1) % M      # 冲突就向后挪一格,越界绕回 0
if table[pos] is None:
    table[pos] = s          # 找到空位,插入
else:
    pass                   # 键已存在,什么也不做
```

手算一遍: $M = 7$,按序插入 7、15、8、1($h(x) = x \bmod 7$): $7 \rightarrow 0$; $15 \rightarrow 1$; $8 \rightarrow 1$ 冲突试 2,空,落位; $1 \rightarrow 1$ 冲突试 2 仍冲突,试 3,落位。最终下标 0-3 依次是 7、15、8、1:一次聚集——三键挤成一团,后面的键要从 1 一路试到 4。

5.3.2 平方探测

平方探测把步长改成平方:第 i 次冲突后尝试 $pos + i^2$,探测序列从一开始就跳开,避免连续占用。

示例 5-5 平方探测插入

```
# 平方探测:第 i 次冲突后尝试 pos + i^2
pos = hash_str(s, M)
i = 0
while table[pos] is not None and table[pos] != s:
    i += 1
    pos = (pos + i * i) % M    # 探测序列:pos+1, pos+4, pos+9, ...
if table[pos] is None:
    table[pos] = s
# 定理(教材 5.4 节):表大小为素数且装填因子  $\lambda < 1/2$  时,
# 平方探测总能找到空单元,且最多探测一半的表长
```

教材定理:表大小是素数且 $\lambda < 1/2$ 时,平方探测总能在最多一半的表长内找到空单元(证明要点:前半段探测位置互不相同)。工程含义: λ 必须死守在半表以下,因此要更早再散列。它消灭了一次聚集,但留下"二次聚集":同位置的键探测序列完全相同。

5.3.3 双散列

双散列连二次聚集也消灭:步长由第二个散列函数算出,每个键拥有专属探测节奏。常见选择:步长 = $M2 - (h(x) \bmod M2)$, $M2$ 取比 M 小的素数。

示例 5-6 双散列插入

```
# 双散列:冲突时以第二个散列函数的值做步长
def hash_str2(s, M):          # 第二散列:常用  $M2 = (h \bmod M2)$ ,  $M2$  为素数
    return M - 1 - (hash_str(s, M) % (M - 1))  # 值域  $1..M-1$ , 步长不为 0

pos = hash_str(s, M)          # 主散列:决定初始位置
step = hash_str2(s, M)        # 第二散列:决定步长, 注意不能为 0
while table[pos] is not None and table[pos] != s:
    pos = (pos + step) % M     # 每次前进 step 格, 而不是 1 格
if table[pos] is None:
    table[pos] = s
```

5.3.4 三种方案对比与惰性删除

方案	探测序列	代价
线性探测	顺次 +1	一次聚集, λ 高时退化严重; 实现最简单
平方探测	+1, +4, +9, ...	无一次聚集, 有二次聚集; 需素数表且 $\lambda < 1/2$
双散列	步长由第二个散列决定	无聚集, 但要多算一次散列函数

⚠ 误区: 开放定址表不能物理删除

线性探测里删掉一个元素若直接置空, 会把探测链拦腰截断: 后续依赖它"借道"的元素从此再也找不到。教材做法是 **惰性删除**: 标记 deleted, 查找跳过、插入复用、定期再散列清标记。注意 deleted 单元不算"已占用", 装填因子计算要把它们排除。

§5.4 再散列

5.4.1 什么时候需要再散列

装填因子涨过阈值后冲突率陡增, 平均链长(或探测次数)失控。唯一的办法是 **再散列**: 表扩容(通常翻倍取素数), 旧元素用新表大小重新散列。插入时检查 λ , 超阈值触发。工程阈值: 分离链接约 0.75, 开放定址约 0.5。

5.4.2 实现: 翻倍、取素数、重插

示例 5-7 再散列: 给分离链接表扩容

```
# 再散列: 装填因子超过阈值就把表翻倍并重新插入全部元素
def rehash(self):
    if self.load_factor < 0.75:
        return          #  $\lambda$  未超标则不做
    old_buckets = self.buckets  # 先备份旧桶
    new_m = next_prime(len(old_buckets) * 2)  # 新表大小: 翻倍后取素数
    self.buckets = [[] for _ in range(new_m)]
    self.size = 0
    for b in old_buckets:
        for x in b:
            self.insert(x)      # 会用新表大小重算散列
# 单次再散列  $O(n)$ , 摊到每次插入上仍是  $O(1)$ ——摊还分析见 §11
```

最后是"重新插入"而非"搬过去": 桶号 = $\text{hash}(x) \bmod \text{新表大小}$, 每个元素都得重算——"再散列"之名由此而来。

5.4.3 摊还代价

单次再散列 $O(n)$,但把账摊到 n 次插入上:每次扩容后空间翻倍,平均下来 每次插入摊还 $O(1)$ 。这与第 3 章 Python list 扩容的摊还分析同构,区别是散列表翻倍后元素必须重插(不能拷贝)。

实现建议:dict 的扩容不需要你操心

CPython 的 dict 在装填因子超过 $2/3$ 时自动扩容(resize),并自动把旧键值对重新散列——这正是教材讲的再散列,只是你感知不到。手写表时要自己写 rehash(示例 5-7);用 dict 时记得:插入 10 万条记录前可用 `d = {}` 直接建,CPython 内部会平滑扩容,无需预分配。

§5.5 可扩散列(选读)

5.5.1 散列与磁盘的冲突

前面所有方案都假设表在内存。数据量大到必须存磁盘时,一次磁盘 I/O 是内存访问的几十万倍,而散列函数均匀分布意味着每次查找都可能访问不同页,随机 I/O 会让性能崩溃。可扩散列(extendible hashing)专治“键太多,一页装不下”。

5.5.2 目录、页与局部分裂

可扩散列分两层:内存里的 目录 (directory,大小为 2^d 的数组)和一组磁盘 页 (page,一页容纳若干键)。查找:取散列值的前 d 位定位目录项,读目录项指向的页。插入时若页满就分裂:满页一分为二,目录规模翻倍(d 加一)。只有满的那一页被拆,其它页原封不动,这叫 局部分裂 ——表随数据增长逐页扩张,而非整体重建。

对照再散列:整表翻倍、全部重插(摊还 $O(1)$);可扩散列只分裂满页 + 目录翻倍,其它页不动,且只有一页 I/O。代价:目录占内存,可能有两页指向同一目录区段(浪费)。数据库与文件系统的索引设计正是这一思想的生产级版本。

§5.6 标准库中的散列结构

5.6.1 dict:Python 内建的散列表

dict 就是一张工程级的散列表:键不可变可哈希、查找平均 $O(1)$ 、自动扩容(§5.4)。两个 Python 特有的行为要说清楚:3.7+ 保证按插入顺序遍历 (与 C++ unordered_map 的无序相反);`d[k]` 在键不存在时抛 `KeyError`(与 C++ operator[] 自动插入恰好相反)。

示例 5-8 dict 基本用法:对单词计数

```
# dict:Python 内建的散列表,键值对即用即走
cnt = {}                                # 空字典
for w in words:
    cnt[w] = cnt.get(w, 0) + 1          # get:取不到返回默认值 0

# 三种“查”的姿势
if w in cnt:                            # 只想知道在不在:in 成员判断
    n = cnt[w]                          # 直接下标:键不存在会 KeyError!
n = cnt.get(w, 0)                       # get:不存在返回默认值,不抛异常

# 遍历:3.7+ 按插入顺序
for k, v in cnt.items():
    print(k, v)
```

⚠ 误区:可变对象不能做键

dict 要求键可哈希,而可哈希的前提是"不可变":list、dict、set 可变,不能当键;`d[[1, 2]] = "x"` 直接抛 `TypeError: unhashable type: 'list'`。tuple 只要内部元素都不可变就可哈希:`d[(1, 2)]` 合法。若你真需要"列表键",把 list 转成 tuple 再用——这是 Python 里最常见的散列坑之一。

5.6.2 可哈希:什么对象能当键

"可哈希"就是"有散列函数"的 Python 说法。内置类型里,整数、浮点、字符串、tuple 可哈希;list、dict、set 不可哈希。自定义类的实例默认可哈希(按对象地址散列,与 `==` 一致),这是 Python 的省心之处——不需要像 C++ 那样写函数对象:

示例 5-9 可哈希规则与一行建表

```
# 可哈希:不可变对象才能当键,list 不行,tuple 可以
d = {}
d["name"] = "Ada"           # str 可哈希
d[(1, 2)] = "point"         # tuple 可哈希
# d[[1, 2]] = "x"           # TypeError: unhashable type: 'list'

# 一行建表:字典推导式
freq = {w: words.count(w) for w in set(words)}

# 更高效的默认值工厂
from collections import defaultdict, Counter
cnt = defaultdict(int)      # 访问不存在的键自动给 0
cnt["hello"] += 1
top3 = Counter(words).most_common(3) # 词频排序,取前 3
```

5.6.3 常用写法:推导、defaultdict 与 Counter

词频统计是 dict 最经典的应用,工程上有三种递进写法:`get + 1` (示例 5-8)、`defaultdict(int)` (少一行判断)、`Counter(words)` (一行全部完成,还带 `most_common` 排序)。选型建议:

需求	写法	原因
普通查值、计数	dict + get	无额外依赖,语义直白
频繁"取不到就设默认"	defaultdict	省掉显式判断
计数 + 频率排序	Counter	most_common 一行搞定
需要按键有序遍历	sorted(d)	无内建有序字典,排序后遍历

注意 dict 与 C 的关系:你写的 `dict` 字面量,底层就是一张散列表;理解本章的 `λ`、再散列,就是理解 dict 在"满"的时候做了什么。

§5.7 手写与标准库并排:单词统计

5.7.1 需求:统计文章词频

统计一篇文章里每个单词的出现次数("the 出现 1800 次"之类)。这是散列表的教科书级应用,也是章末实验的前身。同一需求,两条路线各写一遍,并排对比。

5.7.2 手写版实现

示例 5-10 手写版:桶里放 (单词, 次数) 元组

```
# 手写版:桶里放 (单词, 次数) 元组,统计每个单词的出现次数
class WordCounter:
    def __init__(self, m):
        self.buckets = [[] for _ in range(next_prime(m))]
        self.distinct = 0

    def add(self, w):
        b = self.buckets[self._hash(w)]
        for i, (key, cnt) in enumerate(b):
            if key == w:
                b[i] = (key, cnt + 1)    # 已存在:计数 +1
                return
        b.append((w, 1))                # 新词:插到桶尾
        self.distinct += 1

    def count(self, w):
        for key, cnt in self.buckets[self._hash(w)]:
            if key == w:
                return cnt
        return 0

    def _hash(self, s):
        h = 0
        for c in s:
            h = h * 37 + ord(c)
        return h % len(self.buckets)

def split_words(text):
    return re.findall(r"[a-z0-9]+", text.lower())

def main():
    text = "the cat and the dog and the cat"
    wc = WordCounter(16)                # 表大小 16 → 素数 17
    for w in split_words(text):
        wc.add(w)
    print("the:", wc.count("the"))      # 3
    print("cat:", wc.count("cat"))      # 2
    print("不同单词:", wc.distinct)     # 4

if __name__ == "__main__":
    main()
```

5.7.3 标准库版实现

示例 5-11 标准库版:同一需求,代码量减到 1/5

```
# 标准库版:Counter 一行完成全部计数
import re
from collections import Counter

text = "the cat and the dog and the cat"
words = re.findall(r"[a-z0-9]+", text.lower())
cnt = Counter(words)          # 一次扫描完成全部计数

print("the:", cnt["the"])      # 3
print("cat:", cnt["cat"])      # 2
print("不同单词:", len(cnt))    # 4
print(cnt.most_common(2))      # [('the', 3), ('cat', 2)]
```

5.7.4 对照结论

C 的写法	Python 的写法
结构体数组存 (word, cnt), 每次查找线性扫描 $O(n)$	dict / Counter: 一次扫描完成, 平均 $O(1)$
自己写字符串比较、手动扩容、手动释放	== 内置, 自动扩容, 垃圾回收

调试技巧: 手写版与库版互相验算

手写数据结构最常见的 bug 是“错得整齐”: 两处实现犯同一个错, 测试却全绿。对策是章末实验的做法——同一份数据喂给手写表和 dict, 抽查若干键的词频必须一致。这个“双实现互验”的习惯, 写红黑树、堆时都用得上。

本章复杂度速查表

口诀: 查找/插入/删除平均 $O(1)$, 最坏 $O(n)$; 代价是放弃按键顺序。以下结论两版通用。

操作	数据结构/算法	平均	最坏
查找	散列表(分离链接)	$O(1)$	$O(n)$
插入	散列表(分离链接)	$O(1)$	$O(n)$
删除	散列表(分离链接)	$O(1)$	$O(n)$
查找	散列表(开放定址)	$O(1)$	$O(n)$
插入	散列表(开放定址)	$O(1)$	$O(n)$
删除(惰性)	散列表(开放定址)	$O(1)$	$O(n)$
散列函数	整数取模	$O(1)$	—
散列函数	字符串 37 倍多项式(键长 L)	$O(L)$	—
再散列	单次再散列(翻倍重插)	$O(n)$	$O(n)$
连续插入	含再散列的 n 次插入	摊还 $O(1)$ /次	$O(n)$ /次
查找/插入/删除	dict / set(内建散列表)	$O(1)$	$O(n)$
按插入序遍历	dict(3.7+)	$O(n)$	$O(n)$
按键有序遍历	sorted(d)(无内建有序字典)	$O(n \log n)$	$O(n \log n)$
探测次数	线性/平方探测, $\lambda \approx 0.5$	≈ 1.5 次	$O(n)$
词频统计	手写散列表 / dict / Counter	$O(n)$	$O(n^2)$

最小必会清单

- 能说出 dict 是内建散列表:查找平均 $O(1)$, 3.7+ 保证按插入顺序遍历。
- 能手写取模散列,并知道 Python 的 % 是向下取整(负数余数非负,与 C 不同)。
- 能手写 37 倍多项式字符串散列,解释"为什么先累加、最后取模";整数任意精度无溢出。
- 能说出表大小必须取素数(手写实现时),并给出合数表大小的反例。
- 能说出装填因子 $\lambda = N/M$ 的含义与经验阈值(分离链接 ≈ 0.75 ,平方探测 < 0.5)。
- 能实现分离链接散列表(list of list)的 insert / find / remove。
- 能说出开放定址三种方案的探测序列、聚集现象与适用条件。
- 能解释惰性删除的必要性(物理删除会切断探测链)。
- 能实现再散列,说出"每次插入摊还 $O(1)$ ";知道 dict 自动扩容做的是同一件事。
- 能判断什么对象可哈希:list/dict/set 不能当键,tuple 可以;d[k] 不存在抛 KeyError,get 返回默认值。
- 能用 dict + get、defaultdict、Counter 三种写法做词频统计,知道各自适用场景。

自测题

Q 1. 表大小 $M = 7$,散列函数 $h(x) = x \bmod 7$,用分离链接法依次插入 13、27、41、55。画出各桶的链表,并计算装填因子 λ 与平均链长。

✓ 参考答案

13、27、41、55 除以 7 的余数全是 6——四键全落 6 号桶(最坏情况的构造方式)。 $\lambda = 4/7 \approx 0.57$,平均链长 4/7,但 6 号桶链长 4。

Q 2. 逐字符取模与最后取模,在 $M = 37$ 和 $M = 36$ 时各会怎样退化?

✓ 参考答案

$M = 37: (h \cdot 37 + c) \bmod 37 = c \bmod 37$,退化为"只看最后一个字符"; $M = 36: 37 \equiv 1 \pmod{36}$,每个字符权重都是 1,"abc" 与 "cba" 同值。根治:先累加、最后取一次模。

Q 3. 表大小 $M = 11$,线性探测依次插入 3、14、25、8、19,给出最终表内容与探测过程。

✓ 参考答案

3→3;14→3 冲突→4;25→3 冲突→4 冲突→5;8→8;19→8 冲突→9。最终下标 3、4、5、8、9 依次为 3、14、25、8、19。14、25 挤成连续块,正是"一次聚集"。

Q 4. 开放定址表里删除元素为什么不能直接置空?工程上怎么处理?

✓ 参考答案

直接置空会把探测链拦腰截断:后续键靠被删元素"借道",探测序列在此断掉,再也找不到。工程做法是惰性删除:标记 deleted,查找跳过、插入复用、定期再散列清标记。

Q 5. Python 里为什么 $[1, 2]$ 不能做 dict 的键,而 $(1, 2)$ 可以? $d[k]$ 与 $d.get(k, 0)$ 在键不存在时各是什么行为?

✓ 参考答案

list 可变,不满足可哈希的前提,做键抛 `TypeError: unhashable type`;tuple 不可变,可哈希。 $d[k]$ 键不存在时抛 `KeyError`; $d.get(k, 0)$ 返回默认值 0,不抛异常——只读查询优先用 `get`。

章末实验题:散列表单词统计对比实验

实验 5:散列表单词统计对比实验

实验目标:手写分离链接散列表,与 dict 双路线统计英文文本词频,用计时数据验证装填因子 λ 对性能的影响。

实验要求:

- 任务 1:实现分离链接散列表(键为字符串):37 倍多项式散列、素数表大小、add/count 操作(覆盖 §5.1 散列函数与素数表、§5.2 分离链接法)。
- 任务 2:用自写的表统计文本,输出单词总数、不同单词个数、装填因子 λ (覆盖 §5.1.4 装填因子)。
- 任务 3:用 dict 重做统计,抽查 "the"、"hash"、"hello" 三个词,两套结果必须一致(覆盖 §5.6 标准库;兼作手写版正确性验证)。
- 任务 4:重复插入 30 遍,改变表大小 $M = 8/16/64/256$ (实际素数 $11/17/67/257$),每种测 3 次取最快;观察 λ 越大耗时越高,体会再散列的动机(覆盖 §5.2.3 复杂度、§5.4 再散列)。
- 任务 5:健壮性——无命令行参数、文件打不开、文本为空都要给出友好提示(覆盖边界处理与工程习惯)。

实验环境:Python 3。运行:python lab5.py sample.txt(任意英文文本,词越多对比越明显)。

第一步 写工具函数:next_prime 定素数表大小;split_words 切词(字母数字连续成词,转小写,其余字符当分隔)。先用 3-5 个单词手算验证散列。

第二步 写 HashTable:构造函数用 next_prime 定桶数;hash 用"先累加、最后取模";add 找到键计数 +1,否则插入新键。写完后用小文本对照字典检查词频。

第三步 后写计时:任务 4 对每个 M 重复 3 次取最快,插入重复 30 遍放大差异;跑前先确认 λ 合理(如 208 个不同单词、 $M=257$ 时 $\lambda \approx 0.81$),再做比较。

✅ 参考答案(完整可运行程序,分三段)

实验 5-1 参考答案(第一部分:工具函数与切词)

```
# lab5.py — 散列表单词统计对比实验(第一部分:工具函数)
import re
import sys
import time
from collections import defaultdict

def next_prime(n):
    """返回大于等于 n 的最小素数。"""
    def is_prime(x):
        if x < 2:
            return False
        d = 2
        while d * d <= x:
            if x % d == 0:
                return False
            d += 1
        return True
    while not is_prime(n):
        n += 1
    return n

def split_words(text):
    """任务 5:切分单词—字母/数字连续成词(正则),其余字符跳过。"""
    return re.findall(r"[a-z0-9]+", text.lower())
```

实验 5-1 参考答案(第二部分:手写分离链接散列表)

```
# ----- 任务 1:分离链接散列表(字符串键,37 倍散列,素数表) -----
class HashTable:
    """桶数组是 list of list,一个桶就是一条链表。"""

    def __init__(self, m):
        self.buckets = [[] for _ in range(next_prime(m))]
        self.size = 0

    def _hash(self, s):
        h = 0
        for c in s:
            h = h * 37 + ord(c)  # 全程不取模,最后再 % 表大小(见 §5.1.3)
        return h % len(self.buckets)

    def add(self, s):
        b = self._hash(s)
        for i, (key, cnt) in enumerate(self.buckets[b]):
            if key == s:
                self.buckets[b][i] = (key, cnt + 1)
                return
        self.buckets[b].append((s, 1))
        self.size += 1

    def count(self, s):
        for key, cnt in self.buckets[self._hash(s)]:
            if key == s:
                return cnt
        return 0

    @property
    def load_factor(self):
        return self.size / len(self.buckets)
```

实验 5-1 参考答案(第三部分:main 与四种表大小对比)

```

def main():
    if len(sys.argv) < 2:
        print("用法: python lab5.py <文本文件>")
        return
    try:
        with open(sys.argv[1], encoding="utf-8") as f:
            text = f.read()
    except OSError as e:
        print("错误: 无法打开文件:", e)      # 任务 5:文件打不开
        return
    words = split_words(text)
    if not words:
        print("警告: 文本为空或没有合法单词,无法统计")
        return
    print(f"单词总数(含重复): {len(words)}")

    ht = HashTable(256)                      # 任务 2:手写表统计
    for w in words:
        ht.add(w)
    print(f"手写表: 不同单词 {ht.size} 个,装填因子  $\lambda$ ={{ht.load_factor:.2f}}")

    mp = defaultdict(int)                   # 任务 3:标准库 dict 重做
    for w in words:
        mp[w] += 1
    print(f"dict: 不同单词 {len(mp)} 个")
    for w in ("the", "hash", "hello"):
        print(f"  词 {w!r}: 手写表={{ht.count(w)} dict={{mp[w]}}")

    print("任务4: 表大小(素数)不同  $\rightarrow$   $\lambda$  不同  $\rightarrow$  插入耗时(3 次取最快):")
    for m in (8, 16, 64, 256):
        best = float("inf")
        for _ in range(3):                  # 重复 3 次取最快,减少系统噪声
            t = HashTable(m)
            t0 = time.perf_counter()
            for _ in range(30):              # 重复 30 遍放大差异
                for w in words:
                    t.add(w)
            best = min(best, (time.perf_counter() - t0) * 1000)
        print(f"  M={{len(t.buckets)}  $\lambda$ ={{t.load_factor:.2f}} 耗时 {{best:.2f}} ms")
    print("观察:  $\lambda$  越大平均链表越长,耗时越高——这就是再散列($5.4)的动机。")

if __name__ == "__main__":
    main()

```

编译运行命令与真实输出(sample.txt 为 7 段文章拼成,17520 词、208 个不同单词):

■ 实验 5-1 输出样例(本机实测,数值随机器与负载略有波动,趋势稳定)

```
$ python lab5.py sample.txt
```

单词总数(含重复): 17520

手写表: 不同单词 208 个, 装填因子 $\lambda=0.81$

dict: 不同单词 208 个

词 'the': 手写表=1800 dict=1800

词 'hash': 手写表=120 dict=120

词 'hello': 手写表=120 dict=120

任务4: 表大小(素数)不同 $\rightarrow \lambda$ 不同 $\rightarrow$ 插入耗时(3 次取最快):

M=11 $\lambda=18.91$ 耗时 288.13 ms

M=17 $\lambda=12.24$ 耗时 255.71 ms

M=67 $\lambda=3.10$ 耗时 195.66 ms

M=257 $\lambda=0.81$ 耗时 184.79 ms

观察: λ 越大平均链表越长, 耗时越高——这就是再散列(\$5.4)的动机。

```
$ python lab5.py empty.txt
```

警告: 文本为空或没有合法单词, 无法统计

设计说明:程序分四层, 每层对应一个知识点。散列函数写成"先累加、最后取模"的 37 倍多项式(\$5.1.3), 表大小交给 next_prime(\$5.1.2)——本章最易写错的两处, 用真实数据检验了正确性。桶用 list of list 实现分离链接(\$5.2), 桶内元组携带键与计数, 一次遍历完成"查重或累加"。任务 3 的抽查是刻意的工程习惯: 手写实现与 dict 输出一致, 才敢相信手写版正确。任务 4 把 M 从 11 调到 257, λ 从 18.9 降到 0.81, 耗时从 288 ms 降到 185 ms——M=11 时平均每链 1589 个词, 查找已接近线性扫描, 这是" λ 过高 $\rightarrow$ 性能塌方 $\rightarrow$ 再散列"的实证。计时 3 次取最快 + 重复 30 遍, 压掉系统调度噪声。切词规则决定了"单词"的定义, 是统计程序必须显式声明的约定。真实 dict 的自动扩容正是示例 5-7 再散列的生产级版本。

下一章预告: 第6章 优先队列(堆)——基于数组的完全二叉树; 建堆与删除最小将使用第 3 章的 list, 堆的插入(上滤)与删除最小(下滤)是第 7 章堆排序、第 9 章 Dijkstra 最短路径的基石。